Python Functionsتوابع در پایتون

Functionچیست؟

() Functionیک بلوک از کد است که برای انجام یک کار مشخص نوشته می‌شود و فقط زمانی اجرا می‌شود که آن را Call فراخوانی کنیم و به زبان ساده ، تابع یعنی یک قطعه کد که یک کار مشخص انجام می‌دهد و می‌توانیم هر چند بار که خواستیم از آن استفاده کنیم.

مثلاً اگر چند بار بخواهیم پیام زیر را چاپ کنیم:

```
print("Hello Ali")
print("Hello Ali")
print("Hello Ali")
```

می‌توانیم یک تابع بسازیم:

```
def say_hello():
  print("Hello Ali")
```

و هر زمان لازم بود آن را صدا بزنیم:

```
say_hello()
say_hello()
say_hello()
```

ساختFunction

برای ساخت تابع از کلمه کلیدی defاستفاده می‌کنیم

ساختار کلی:

```
def function_name():
  # code
```

مثال:

```
def my_function():
  print("Hello from a function")
```

اینجا تابعی به نام my_functionساخته‌ایم اما هنوز اجرا نشده است

Calling a Function

برای اجرای تابع، نام آن را همراه با ()می‌نویسیم:

```
def my_function():
  print("Hello from a function")
my_function()
```

خروجی:

```
Hello from a function
```

نکته بسیار مهم : این دو با هم فرق دارند

```
my_function
```

و:

```
my_function()
```

دومی تابع را اجرا می‌کند

اجرای چندبارهFunction

یک تابع را می‌توانیم چندین بار فراخوانی کنیم:

```
def say_hello():
  print("Hello")
say_hello()
say_hello()
say_hello()
```

خروجی:

```
Hello
Hello
Hello
```

چرا از Function استفاده می‌کنیم؟

یکی از مهم‌ترین دلایل استفاده از Function جلوگیری از **تکرار کد (Code Repetition)** است

فرض کنید می‌خواهیم سه عدد را با عدد ۱۰ جمع کنیم:

بدون تابع:

```
number1 = 5
result1 = number1 + 10
print(result1)
number2 = 20
result2 = number2 + 10
print(result2)
number3 = 50
result3 = number3 + 10
print(result3)
```

کد تکراری شده است

با Function:

```
def add_10(number):
    return number + 10
print(add_10(5))
print(add_10(20))
print(add_10(50))
```

خروجی:

```
15
30
60
```

Functionبا Parameter

گاهی می‌خواهیم هنگام فراخوانی تابع، اطلاعاتی را به آن بدهیم

مثلاً:

```
def greet(name):
    print("Hello", name)
```

حالا:

```
greet("Ali")
greet("Sara")
greet("Reza")
```

خروجی:

```
Hello Ali
Hello Sara
Hello Reza
```

در اینجا nameیک Parameterاست

Parameterو Argument

در تعریف تابع:

```
def greet(name):
```

name → **Parameter**

هنگام فراخوانی:

```
greet("Ali")
```

"Ali" → **Argument**

یعنی:

Parameter → متغیری که هنگام تعریف تابع می‌نویسیم

Argument → مقداری که هنگام فراخوانی به تابع می‌دهیم

چندParameter

یک Function می‌تواند چند Parameter داشته باشد:

```
def add(a, b):
   print(a + b)
```

استفاده:

```
add(5, 3)
add(10, 20)
```

خروجی:

```
8
30
```

استفاده از return

تا اینجا تابع ما نتیجه را printمی‌کرد:

```
def add(a, b):
   print(a + b)
```

اما گاهی می‌خواهیم نتیجه را به برنامه برگردانیم

در این حالت از returnاستفاده می‌کنیم:

```
def add(a, b):
   return a + b
```

حالا:

```
result = add(5, 3)
print(result)
```

خروجی:

```
8
```

تفاوت print و return: این تفاوت بسیار مهم است

با print

```
def add(a, b):
    print(a + b)
```

تابع نتیجه را نمایش می‌دهد

با return

```
def add(a, b):
    return a + b
```

تابع نتیجه را برمی‌گرداند و می‌توانیم آن را ذخیره یا در محاسبات دیگر استفاده کنیم

مثلاً:

```
def add(a, b):
    return a + b
result = add(10, 20)
print(result * 2)
```

خروجی:

```
60
```

Function بدون Parameter و با Return

تابع الزاماً Parameter ندارد

```
def get_name():
    return "Ali"
```

استفاده:

```
name = get_name()
print(name)
```

خروجی:

Ali

یا مستقیماً:

```
print(get_name())
```

Functionبدون return

اگر تابع returnنداشته باشد، مقدار بازگشتی آن به صورت پیش‌فرض None است

مثلاً:

```
def hello():
   print("Hello")
result = hello()
print(result)
```

خروجی:

```
Hello
None
```

چرا؟ چون ()hello فقط چیزی را چاپ می‌کند و هیچ مقداری با returnبرنمی‌گرداند

returnباعث پایان Function می‌شود وقتی Python به returnمی‌رسد، اجرای Function متوقف می‌شود

```
def test():
   print("A")
   return
   print("B")
test()
```

خروجی:

```
A
```

"B"چاپ نمی‌شود چون return باعث پایان Function شده است

مثال - محاسبه مساحت مربع

```
def square_area(side):
    return side * side
print(square_area(5))
print(square_area(10))
```

خروجی:

```
25
100
```

مثال - زوج یا فرد

با ترکیب Function و if:

```
def check_even(number):
    if number % 2 == 0:
        return "Even"
    else:
        return "Odd"
print(check_even(10))
print(check_even(7))
```

خروجی:

```
Even
Odd
```

مثال -تبدیل Fahrenheit بهCelsius

```
def fahrenheit_to_celsius(fahrenheit):
   return (fahrenheit - 32) * 5 / 9
print(fahrenheit_to_celsius(77))
print(fahrenheit_to_celsius(95))
print(fahrenheit_to_celsius(50))
```

Functionبا چندParameter

مثلاً محاسبه میانگین دو عدد:

```
def average(a, b):
   return (a + b) / 2
result = average(10, 20)
print(result)
```

خروجی:

```
15.0
```

مثال با for

Functionمی‌تواند شامل Loop هم باشد:

```
def print_numbers():
   for i in range(1, 6):
      print(i)
print_numbers()
```

خروجی:

```
1
2
3
```

4

5

Function + List + For

```
def print_fruits(fruits):
    for fruit in fruits:
        print(fruit)
my_fruits = ["apple", "banana", "orange"]
print_fruits(my_fruits)
```

خروجی:

```
apple
banana
orange
```

Function + List + Return

مثلاً محاسبه مجموع اعداد:

```
def calculate_sum(numbers):
    total = 0
    for number in numbers:
        total += number
    return total
numbers = [10, 20, 30, 40]
result = calculate_sum(numbers)
print(result)
```

خروجی:

```
100
```

Function Naming

نام Function تقریباً همان قوانین نام‌گذاری Variable را دارد

صحیح:

```
calculate_sum()
get_name()
print_result()
check_number()
```

نامناسب:

```
calculate sum()   # ×
2calculate()      # ×
```

بهتر است نام تابع توضیح دهد که تابع چه کاری انجام می‌دهد

مثلاً:

```
calculate_average()
check_password()
find_maximum()
convert_temperature()
```

passدر Function

گاهی می‌خواهیم فعلاً یک Function بسازیم ولی هنوز کد آن را ننوشته‌ایم

```
def calculate_salary():
    pass
```

passیعنی فعلاً هیچ کاری انجام نده بعداً می‌توانیم آن را تکمیل کنیم

انواع Function تا اینجا

مثال	نوع Function
def hello():	بدون Parameter ، بدون Return
def greet(name):	با Parameter ، بدون Return
def get_name():	بدون Parameter ، با Return
def add(a, b):	با Parameter ، با Return

مثلاً مهم‌ترین نوع:

```
def add(a, b):
    return a + b
```

تمرین ۱

تابعی به نام hello() بسازید که عبارت زیر را چاپ کند:

Hello Python

تمرین ۲

تابعی به نام greet(name) بسازید که نام شخص را دریافت کند و چاپ کند:

Hello Ali

مثلاً:

```
greet("Ali")
greet("Sara")
```

تمرین ۳

تابعی به نام add(a, b) بنویسید که مجموع دو عدد را return کند

مثلاً:

```
print(add(10, 20))
```

خروجی:

30

تمرین ۴

تابعی بنویسید که یک عدد دریافت کند و مربع آن را برگرداند

```
square(5)
```

خروجی:

```
25
```

تمرین ۵

تابعی به نام is_even(number) بنویسید که بررسی کند عدد زوج است یا فرد

مثلاً:

```
print(is_even(10))
```

خروجی:

```
Even
```

تمرین ۶

تابعی بنویسید که سه عدد دریافت کند و بزرگ‌ترین عدد را بدون استفاده از max() پیدا کند

```
find_max(10, 25, 15)
```

خروجی:

```
25
```

تمرین ۷

تابعی به نام calculate_average() بنویسید که یک List از اعداد دریافت کند و میانگین آنها را برگرداند

مثلاً:

```
numbers = [10, 20, 30, 40]
print(calculate_average(numbers))
```

خروجی:

```
25.0
```

تمرین ۸

تابعی بنویسید که یک List دریافت کند و فقط اعداد زوج را چاپ کند

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]
```

خروجی:

```
2
4
6
8
```

تمرین Calculator

یک برنامه Calculator با Function بسازید

چهار تابع ایجاد کنید:

```
add()
subtract()
multiply()
divide()
```

مثلاً:

```
print(add(10, 5))
print(subtract(10, 5))
print(multiply(10, 5))
print(divide(10, 5))
```

خروجی:

```
15
5
50
2.0
```

تمرین Student Grade

تابعی به نام check_grade(score) بسازید

قوانین:

90 - 100 → A

80 - 89 → B

70 - 79 → C

60 - 69 → D

Below 60 → F

مثلاً:

```
print(check_grade(85))
```

خروجی:

B

پروژه Student Management System

یک برنامه ساده برای مدیریت دانش‌آموزان طراحی کنید برنامه باید حداقل این Function ها را داشته باشد:

add_student()

calculate_average()

check_grade()

find_best_student()

مثلاً اطلاعات دانش‌آموزان:

```
students = {
   "Ali": 18,
   "Sara": 15,
   "Reza": 19,
   "Maryam": 17
}
```

برنامه بتواند:

۱. دانش‌آموز جدید اضافه کند
۲. نمره را دریافت کند
۳. میانگین نمرات را محاسبه کند
۴. وضعیت نمره را مشخص کند
۵. بهترین دانش‌آموز را پیدا کند

Python Function Arguments آرگومان‌ها در توابع پایتون

تا اینجا یاد گرفتیم که می‌توانیم یک Function بسازیم:

```
def greet():
   print("Hello")
```

اما گاهی لازم است هنگام اجرایFunction ، اطلاعاتی را به آن بدهیم به این اطلاعات Argumentمی‌گوییم

Argumentچیست؟

فرض کنید تابع زیر را داریم:

```
def greet(name):
   print("Hello", name)
```

حالا:

```
greet("Ali")
```

در اینجا "Ali"یک Argumentاست

خروجی:

```
Hello Ali
```

می‌توانیم مقدارهای مختلفی به تابع بدهیم:

```
greet("Ali")
greet("Sara")
greet("Reza")
```

خروجی:

Hello Ali

Hello Sara

Hello Reza

Parameterو Argument

این دو مفهوم را باید از هم جدا کنیم در تعریفFunction

def greet(name):

name → **Parameter**

در زمان فراخوانی:

greet("Ali")

"Ali" → **Argument**

یک مثال دیگر

def add(a, b):

  return a + b

add(10, 20)

اینجا:

a , b → Parameters

10 , 20 → Arguments

چندArgument

یک Function می‌تواند چند Parameter داشته باشد

def full_name(first_name, last_name):

  print(first_name, last_name)

فراخوانی:

full_name("Ali", "Pourfereydoon")

خروجی:

Ali Pourfereydoon

می‌توانیم چند بار آن را اجرا کنیم:

```
full_name("Ali", "Pourfereydoon")
full_name("Sara", "Ahmadi")
full_name("Reza", "Karimi")
```

تعداد Arguments باید درست باشد

اگر Function دو Parameter داشته باشد، معمولاً باید دو Argument دریافت کند

```
def add(a, b):
    print(a + b)
```

صحیح:

```
add(10, 20)
```

اما این اشتباه است:

```
add(10)
```

چون bمقدار ندارد

و این هم اشتباه است:

```
add(10, 20, 30)
```

چون Function فقط دو Parameter دارد

Default Parameter

گاهی می‌خواهیم اگر کاربر Argument وارد نکرد، Functionیک مقدار پیش‌فرض داشته باشد

مثلاً:

```
def greet(name="Friend"):
    print("Hello", name)
```

اگر نام بدهیم:

```
greet("Ali")
```

خروجی:

```
Hello Ali
```

اگر نام ندهیم:

```
greet()
```

خروجی:

```
Hello Friend
```

مثال کاربردی Default Parameter

```
def welcome(name="Student"):
    print("Welcome", name)
welcome("Ali")
welcome("Sara")
welcome()
```

خروجی:

```
Welcome Ali
Welcome Sara
Welcome Student
```

Keyword Arguments

در حالت عادی:

```
def person(name, age):
    print(name, age)
```

می‌توانیم بنویسیم:

```
person("Ali", 43)
```

این حالت Positional Argument است اما می‌توانیم مشخص کنیم هر مقدار مربوط به کدام Parameter است:

```
person(name="Ali", age=43)
```

این می‌شود **Keyword Argument**

مزیتKeyword Arguments

در Keyword Arguments ترتیب اهمیت ندارد

مثلاً:

```
def person(name, age):
   print(name, age)
```

هر دو کد صحیح هستند:

```
person(name="Ali", age=43)
```

و:

```
person(age=43, name="Ali")
```

چون خودمان مشخص کرده‌ایم:

```
name = "Ali"
age = 43
```

Positional Arguments

اگر از نام Parameter استفاده نکنیم، Argumentها بر اساس موقعیت به‌Parameter ها متصل می‌شوند

```
def person(name, age):
   print(name, age)
person("Ali", 43)
```

Pythonاین‌طور در نظر می‌گیرد:

```
name ← "Ali"
age  ← 43
```

اما اگر ترتیب را عوض کنیم:

```
person(43, "Ali")
```

Pythonتصور می‌کند:

name ← 43

age ← "Ali"

بنابراین نتیجه اشتباه خواهد بود

تفاوت Positional وKeyword

Positional

person("Ali", 43)

ترتیب مهم است

Keyword

person(name="Ali", age=43)

ترتیب مهم نیست

ترکیب Positional وKeyword

می‌توانیم این دو را با هم استفاده کنیم

```
def person(name, age, city):
   print(name, age, city)
```

مثلاً:

person("Ali", age=43, city="Istanbul")

این صحیح است

اما این اشتباه است:

person(name="Ali", 43, city="Istanbul")

چون Positional Argumentباید قبل از Keyword Argument قرار بگیرد

ارسال List بهFunction

Argumentفقط لازم نیست String یا Number باشد می‌توانیم List هم ارسال کنیم

```
def print_fruits(fruits):
   for fruit in fruits:
```

```
    print(fruit)
fruits = ["apple", "banana", "orange"]
print_fruits(fruits)
```

خروجی:

```
apple
banana
orange
```

ارسال Dictionary به Function

حتی Dictionary هم می‌توانیم ارسال کنیم:

```
def show_person(person):
  print("Name:", person["name"])
  print("Age:", person["age"])
student = {
  "name": "Ali",
  "age": 43
}
show_person(student)
```

خروجی:

```
Name: Ali
Age: 43
```

این مثال برای دانش‌آموزان مهم است چون نشان می‌دهد Function می‌تواند باCollection ها هم کار کند

Returnهمراه باArguments

Argumentsو returnمعمولاً با هم استفاده می‌شوند

```
def add(a, b):
  return a + b
```

حالا:

```
result = add(10, 20)
print(result)
```

خروجی:

```
30
```

Function .می‌تواند List برگرداند

```
def get_fruits():
    return ["apple", "banana", "orange"]
fruits = get_fruits()
print(fruits)
```

خروجی:

```
['apple', 'banana', 'orange']
```

و می‌توانیم به عناصر آن دسترسی داشته باشیم:

```
print(fruits[0])
print(fruits[1])
```

Functionمی‌تواند Dictionary برگرداند

```
def get_student():
    return {
        "name": "Ali",
        "age": 20
    }
student = get_student()
print(student["name"])
print(student["age"])
```

خروجی:

Ali

20

پس returnفقط برای Number نیست

Functionمی‌تواند برگرداند:

int

float

string

list

tuple

dictionary

set

Positional-Only Arguments

این قسمت کمی پیشرفته‌تر است می‌توانیم مشخص کنیم که یک Parameter فقط به صورت Positional دریافت شود

برای این کار از /استفاده می‌کنیم

```
def greet(name, /):
   print("Hello", name)
```

صحیح:

```
greet("Ali")
```

اما:

```
greet(name="Ali")
```

خطا می‌دهد

Keyword-Only Arguments

برعکس حالت قبلی، می‌توانیم بگوییم یک Parameter فقط باید به صورت Keyword ارسال شود

برای این کار از *استفاده می‌کنیم:

```
def greet(*, name):
   print("Hello", name)
```

صحیح:

```
greet(name="Ali")
```

اما:

```
greet("Ali")
```

خطا می‌دهد یعنی:

ترکیب Positional-Only وKeyword-Only

می‌توانیم هر دو را در یک Function داشته باشیم:

```
def calculate(a, b, /, *, c, d):
   return a + b + c + d
```

فراخوانی صحیح:

```
result = calculate(5, 10, c=15, d=20)
print(result)
```

خروجی:

```
50
```

جمع‌بندیArguments

نوع	مثال	توضیح
Positional	func(10, 20)	ترتیب مهم است
Keyword	func(a=10, b=20)	نام Parameter مشخص است
Default	def func(x=10)	مقدار پیش‌فرض
Positional-Only	def func(x, /)	فقطPositional
Keyword-Only	def func(*, x)	فقطKeyword

تمرین ۱

تابعی به نام greet(name)بنویسید که نام شخص را دریافت کرده و پیام زیر را چاپ کند:

Hello Ali

مثلاً:

```
greet("Ali")
greet("Sara")
```

تمرین ۲

تابعی به نام add(a, b) بنویسید که مجموع دو عدد را چاپ کند

```
add(10, 20)
add(5, 7)
```

تمرین ۳

تابعی به نام multiply(a, b) بنویسید که حاصل‌ضرب دو عدد را return کند

```
result = multiply(5, 4)
print(result)
```

خروجی:

```
20
```

تمرین ۴

تابعی به نام greet(name="Student") بسازید

اگر نام وارد شود:

```
greet("Ali")
```

خروجی:

```
Hello Ali
```

اگر نام وارد نشود:

```
greet()
```

خروجی:

```
Hello Student
```

تمرین ۵

تابع زیر را ایجاد کنید:

```
def student_info(name, age):
```

سپس آن را با Positional Argumentsو Keyword Argumentsفراخوانی کنید

تمرین ۶

تابعی بنویسید که یک List از اعداد دریافت کند و مجموع اعداد را برگرداند

مثلاً:

```
numbers = [10, 20, 30, 40]
print(calculate_sum(numbers))
```

خروجی:

```
100
```

تمرین ۷

تابعی بنویسید که یک Dictionary مربوط به دانش‌آموز را دریافت کند:

```
student = {
   "name": "Ali",
   "age": 20,
   "score": 18
}
```

و اطلاعات آن را چاپ کند

تمرین ۸

تابعی بنویسید که سه عدد دریافت کند و بزرگ‌ترین عدد را بدون استفاده از max()برگرداند

```
find_max(10, 25, 15)
```

خروجی:

```
25
```

تمرین ۹

چهار Function بسازید:

add(a, b)

subtract(a, b)

multiply(a, b)

divide(a, b)

هر Function باید نتیجه را با returnبرگرداند

مثلاً:

```
print(add(10, 5))
print(subtract(10, 5))
print(multiply(10, 5))
print(divide(10, 5))
```

تمرین ۱۰Student Grade

تابعی ایجاد کنید check_grade(score)

قوانین:

90 - 100 → A

80 - 89 → B

70 - 79 → C

60 - 69 → D

0 - 59 → F

مثلاً:

```
print(check_grade(85))
```

خروجی:

B

Python *args and **kwargs

مشکل چیست؟

قبلاً یاد گرفتیم یک Function می‌تواند تعداد مشخصی Argument داشته باشد:

```
def add(a, b):
   return a + b
```

اینجا تابع دقیقاً دو Argument می‌خواهد:

```
add(10, 20)
```

اما اگر بخواهیم تابعی داشته باشیم که بتواند هر تعداد Argument دریافت کند، چه کار کنیم؟

مثلاً:

```
add(1, 2)
add(1, 2, 3)
add(1, 2, 3, 4, 5)
```

اینجا *argsبه کمک ما می‌آید

چیست؟*args

*argsبه Function اجازه می‌دهد تعداد نامحدودی Positional Argument دریافت کند

```
def my_function(*args):
   print(args)
```

حالا:

```
my_function("Ali", "Sara", "Reza")
```

خروجی:

```
('Ali', 'Sara', 'Reza')
```

نکته بسیار مهم:

داخلFunction ، argsیک Tupleاست

یعنی:

```
def my_function(*args):
  print(type(args))
```

خروجی:

```
<class 'tuple'>
```

چرا اسم آن argsاست؟

در واقع argsاسم خاصی نیست . این کاملاً صحیح است:

```
def test(*numbers):
  print(numbers)
```

و این هم صحیح است:

```
def test(*values):
  print(values)
```

اما در Python یک قرارداد رایج وجود دارد:

*args

دسترسی به عناصر args*

چون argsیک Tuple است، می‌توانیم با Index به عناصر آن دسترسی داشته باشیم:

```
def my_function(*args):
  print(args[0])
  print(args[1])
  print(args[2])
my_function("Ali", "Sara", "Reza")
```

خروجی:

```
Ali
Sara
Reza
```

استفاده از for با *args

می‌توانیم روی argsحلقه اجرا کنیم:

```
def show_names(*args):
    for name in args:
        print(name)
show_names("Ali", "Sara", "Reza")
```

خروجی:

```
Ali
Sara
Reza
```

مثال- جمع چند عدد

بدون :*args

```
def add(a, b):
    return a + b
```

فقط دو عدد می‌توانیم بدهیم

اما با :*args

```
def add(*numbers):
    total = 0
    for number in numbers:
        total += number
    return total
```

حالا:

```
print(add(1, 2))
print(add(1, 2, 3))
print(add(1, 2, 3, 4, 5))
```

خروجی:

3

6

15

پس تابع ما انعطاف‌پذیر شده است

args*با Parameter معمولی

می‌توانیم Parameter معمولی و args*را با هم استفاده کنیم

```
def greet(greeting, *names):
   for name in names:
      print(greeting, name)
```

حالا:

```
greet("Hello", "Ali", "Sara", "Reza")
```

خروجی:

Hello Ali

Hello Sara

Hello Reza

Pythonاین‌طور عمل می‌کند:

greeting → "Hello"

names → ("Ali", "Sara", "Reza")

نکته مهم

Parameterهای معمولی باید قبل از args*قرار بگیرند.

صحیح:

```
def test(name, *args):
   pass
```

اما این ساختار مناسب این کاربرد نیست:

```
def test(*args, name):
    pass
```

در این حالت nameبه صورت Keyword-Only رفتار می‌کند و باید به شکل دیگری فراخوانی شود

kwargsچیست؟**

حالا فرض کنید نمی‌دانیم چند Keyword Argumentقرار است به Function ارسال شود

برای این کار از:

kwargs** استفاده می‌کنیم

مثلاً:

```
def person(**kwargs):
    print(kwargs)
```

حالا:

```
person(name="Ali", age=20, city="Istanbul")
```

خروجی:

```
{'name': 'Ali', 'age': 20, 'city': 'Istanbul'}
```

نکته مهم:

kwargsداخل Function یک Dictionaryاست

چرا اسم kwargs؟

مثل args، کلمه kwargsهم اسم خاصی نیست

مثلاً این هم صحیح است:

```
def person(**data):
    print(data)
```

اما قرارداد رایج Python این است:

kwargs که مخفف: **Keyword Arguments است

دسترسی به kwargs

چون kwargs یک Dictionary است، می‌توانیم با Key به اطلاعات دسترسی داشته باشیم:

```
def person(**kwargs):
    print(kwargs["name"])
    print(kwargs["age"])
person(name="Ali", age=20)
```

خروجی:

```
Ali
20
```

استفاده از for با **kwargs

می‌توانیم روی Dictionary حلقه بزنیم:

```
def show_info(**kwargs):
    for key, value in kwargs.items():
        print(key, ":", value)
show_info(
    name="Ali",
    age=20,
    city="Istanbul"
)
```

خروجی:

```
name : Ali
age : 20
city : Istanbul
```

args*در مقابل **kwargs

ویژگی	*args	**kwargs
نوعArgument	Positional	Keyword
داخلFunction	Tuple	Dictionary
تعداد	نامحدود	نامحدود
مثال	10, 20, 30	name="Ali"

مثال مقایسه‌ای

```
def test(*args):
    print(args)
test(10, 20, 30)
```

نتیجه:

```
(10, 20, 30)
```

اما:

```
def test(**kwargs):
    print(kwargs)
test(name="Ali", age=20)
```

نتیجه:

```
{'name': 'Ali', 'age': 20}
```

استفاده همزمان از *args و **kwargs

می‌توانیم هر دو را در یک Function استفاده کنیم:

```
def my_function(title, *args, **kwargs):
    print("Title:", title)
    print("Arguments:", args)
    print("Keyword Arguments:", kwargs)
```

فراخوانی:

```
my_function(
   "Student",
   "Ali",
   "Sara",
   age=20,
   city="Istanbul"
)
```

تقریباً این اطلاعات را دریافت می‌کنیم:

```
title  → "Student"
args   → ("Ali", "Sara")
kwargs → {
   "age": 20,
   "city": "Istanbul"
}
```

ترتیبParameters

در حالت معمول، ترتیب به این صورت است:

```
def function(regular_parameters, *args, **kwargs):
   ...
```

مثلاً:

```
def student(name, *subjects, **info):
   print("Name:", name)
   print("Subjects:", subjects)
   print("Info:", info)
```

استفاده:

```
student(
    "Ali",
    "Python",
    "Math",
    "Physics",
    age=20,
    city="Istanbul"
)
```

نتیجه مفهومی:

```
name:
Ali
subjects:
("Python", "Math", "Physics")
info:
{
    "age": 20,
    "city": "Istanbul"
}
```

Unpacking چیست؟

*و **فقط در تعریف Function استفاده نمی‌شوند هنگام فراخوانی Function نیز می‌توانیم از آنها برای Unpacking استفاده کنیم

Unpackingیک List با *

فرض کنید:

```
numbers = [10, 20, 30]
```

و Function:

```
def add(a, b, c):
    return a + b + c
```

می‌توانیم بنویسیم:

```
result = add(*numbers)
print(result)
```

**Unpacking Dictionaryبا **

فرض کنید:

```
person = {
    "name": "Ali",
    "age": 20
}
```

و:

```
def show_person(name, age):
    print(name, age)
```

می‌توانیم بنویسیم:

```
show_person(**person)
```

که تقریباً معادل است با:

```
show_person(name="Ali", age=20)
```

در تعریف:Function

def test(*args):

یعنی Argumentsرا جمع کن و داخل یک Tuple قرار بده

در فراخوانی:

test(*numbers) یعنی این Collection را باز کن و عناصر آن را به عنوان Arguments ارسال کن

مثال کامل

```
def student_info(name, *subjects, **details):
    print("Name:", name)
    print("Subjects:")
    for subject in subjects:
        print("-", subject)
    print("Details:")
    for key, value in details.items():
        print(key, ":", value)
student_info(
    "Ali",
    "Python",
    "Math",
    "Physics",
    age=20,
    city="Istanbul"
)
```

خروجی:

```
Name: Ali
Subjects:
- Python
- Math
- Physics
Details:
age : 20
city : Istanbul
```

تمرین ۱

تابعی بنویسید که با استفاده از *argsهر تعداد نام را دریافت کند و آنها را چاپ کند

```
show_names("Ali", "Sara", "Reza", "Maryam")
```

تمرین ۲

تابعی به نام sum_numbers(*numbers)بنویسید که مجموع هر تعداد عدد را محاسبه کند

مثلاً:

```
sum_numbers(1, 2, 3)
```

خروجی:

```
6
```

و:

```
sum_numbers(10, 20, 30, 40, 50)
```

خروجی:

```
150
```

تمرین ۳

تابعی بنویسید که با *argsهر تعداد عدد دریافت کند و بزرگ‌ترین عدد را بدون max()پیدا کند

مثلاً:

```
find_max(10, 25, 7, 40, 15)
```

خروجی:

```
40
```

تمرین ۴

با استفاده از **kwargs تابعی ایجاد کنید که اطلاعات یک شخص را دریافت کند

مثلاً:

```
person(
   name="Ali",
   age=20,
   city="Istanbul"
)
```

و تمام اطلاعات را چاپ کند

تمرین ۵

تابعی ایجاد کنید:

```
student(name, **info)
```

مثلاً:

```
student(
   "Ali",
   age=20,
   major="Computer Engineering",
   city="Istanbul"
)
```

خروجی مناسب چاپ کنید

تمرین ۶

تابعی ایجاد کنید که یک عنوان و هر تعداد نام دریافت کند:

```
greet("Hello", "Ali", "Sara", "Reza")
```

خروجی:

Hello Ali

Hello Sara

Hello Reza

تمرین ۷

تابعی با ساختار زیر ایجاد کنید:

```
def student(name, *courses, **info):
```

مثلاً:

```
student(
   "Ali",
   "Python",
   "Machine Learning",
   "Data Mining",
   age=25,
   city="Istanbul"
)
```

برنامه باید نام، درس‌ها و اطلاعات اضافی را جداگانه نمایش دهد

تمرین ۸ Calculator پیشرفته

تابعی ایجاد کنید:

```
calculate(operation, *numbers)
```

مثلاً:

```
calculate("sum", 10, 20, 30)
```

خروجی:

```
60
```

و:

```
calculate("max", 10, 50, 20, 30)
```

خروجی:

```
50
```

برای انتخاب عملیات می‌توانید از if/elif یا match که قبلاً یاد گرفته‌اید استفاده کنید

Python Scope

Scope چیست؟

Scope یعنی محدوده‌ای که یک متغیر در آن قابل دسترسی است به زبان ساده هر متغیر در Python در یک محدوده مشخص زندگی می‌کند و فقط در همان محدوده یا محدوده‌های مجاز قابل استفاده است

مثلاً:

```
x = 10
print(x)
```

اینجا x خارج از تابع ساخته شده، بنابراین یک Global Variable است

Local Scope

اگر متغیری داخل یک تابع ساخته شود، آن متغیر Local است

```
def my_function():
  x = 100
  print(x)
my_function()
```

خروجی:

```
100
```

اما این کار اشتباه است:

```
def my_function():
  x = 100
my_function()
```

```
print(x)
```

خطا:

```
NameError
```

چون xفقط داخل my_function()وجود دارد

مثال

```
def student():
    name = "Ali"
    print(name)
student()
```

اینجا name → Local Variable

Global Scope

اگر متغیر خارج از تابع ساخته شود، Globalاست.

```
name = "Ali"
def student():
    print(name)
student()
```

خروجی:

```
Ali
```

چون تابع می‌تواند متغیر Global را بخواند

Localو Global با یک نام

```
x = 100
def test():
    x = 200
    print(x)
```

```
test()
print(x)
```

خروجی:

200

100

چرا؟ چون این دو xمتفاوت هستند:

Global x → 100

Local x → 200

Global Keyword

گاهی می‌خواهیم داخل یک تابع مقدار یک متغیر Global را تغییر دهیم

در این حالت از globalاستفاده می‌کنیم

```
x = 100
def change():
    global x
    x = 200
change()
print(x)
```

خروجی:

200

بدون :global

```
x = 100
def change():
    x = 200
```

```
change()
print(x)
```

خروجی:

```
100
```

چون x = 200 یک متغیر Local جدید ایجاد کرده است

یک مثال برای global

فرض کنید تعداد دانش‌آموزان را ذخیره کرده‌ایم:

```
students = 10
def add_student():
    global students
    students += 1
add_student()
add_student()
print(students)
```

خروجی:

```
12
```

اما در برنامه‌های واقعی معمولاً بهتر است تا حد امکان از تغییر مستقیم متغیرهای Global پرهیز کنیم و از پارامترها و return استفاده کنیم

مثلاً:

```
def add_student(number):
    return number + 1
students = 10
students = add_student(students)
print(students)
```

این روش معمولاً تمیزتر است

Function Inside Function

یک تابع می‌تواند داخل تابع دیگری قرار بگیرد

```
def outer():
   x = 100
   def inner():
      print(x)
   inner()
outer()
```

خروجی:

```
100
```

تابع inner()می‌تواند xرا از تابع بیرونی ببیند اینجا xدر محدوده Enclosingقرار دارد

Nonlocal

nonlocalزمانی استفاده می‌شود که یک تابع داخلی بخواهد مقدار متغیری را که در تابع بیرونی ساخته شده تغییر دهد

```
def outer():
   x = 10
   def inner():
      nonlocal x
      x = 20
   inner()
   print(x)
outer()
```

خروجی:

```
20
```

بدون nonlocal:

```
def outer():
```

```
    x = 10
    def inner():
        x = 20
    inner()
    print(x)
outer()
```

خروجی:

10

چون x = 20 در این حالت یک متغیر Local برای inner() می‌شود

تفاوت global و nonlocal

این جدول برای دانش‌آموزان بسیار مفید است:

Keyword	مربوط به کدام Scope ؟	کاربرد
global	Global	تغییر متغیر Global
nonlocal	Enclosing	تغییر متغیر تابع بیرونی
هیچ‌کدام	Local	ایجاد متغیر Local

مثال:

```
x = 10
def outer():
    y = 20
    def inner():
        global x
        nonlocal y
        x = 100
        y = 200
    inner()
```

```
outer()
print(x)
```

خروجی:

```
100
```

و yدر تابع outer()به 200تغییر کرده است

قانونLEGB

Pythonهنگام پیدا کردن یک متغیر، محدوده‌ها را به ترتیب زیر جستجو می‌کند:

L → E → G → B

L — Local

داخل تابع فعلی:

```
def test():
    x = 10
```

E — Enclosing

داخل تابع بیرونی:

```
def outer():
    x = 20
    def inner():
        print(x)
```

G — Global

در سطح اصلی برنامه:

```
x = 30
```

B — Built-in

توابع و نام‌های داخلی:Python

print() ، len()، sum() ، max() ، min()

مثلاً:

```
x = "Global"
def outer():
    x = "Enclosing"
    def inner():
        x = "Local"
        print(x)
    inner()
outer()
```

خروجی:

```
Local
```

چون Python ابتدا Local را پیدا می‌کند

مثال برای فهمLEGB

```
x = "Global"
def outer():
    x = "Enclosing"
    def inner():
        x = "Local"
        print(x)
    inner()
outer()
```

یک نکته مهم دربارهScope

این کد را ببینید:

```
x = 10
```

```
def test():
   print(x)
test()
```

کاملاً درست است

اما:

```
x = 10
def test():
   x = x + 1
test()
```

این کد خطا می‌دهد چرا؟ چون Python با دیدن x = x + 1 در تابع، xرا به عنوان Local در نظر می‌گیرد؛ بنابراین سمت راست هم به دنبال Local x می‌گردد، در حالی که هنوز مقداردهی نشده است

برای تغییر Global باید بنویسیم:

```
x = 10
def test():
   global x
   x = x + 1
test()
print(x)
```

خروجی:

```
11
```

تمرین Local۱

خروجی برنامه را مشخص کنید:

```
def test():
   x = 50
```

```
    print(x)
test()
```

سؤال :خروجی چیست؟

تمرین ۲Global

```
x = 100
def test():
    print(x)
test()
```

سؤال :خروجی چیست و چرا؟

تمرین ۳Local vs Global

```
x = 10
def test():
    x = 20
    print(x)
test()
print(x)
```

سؤال :خروجی دو printچیست؟

تمرین ۴Global Keyword

کد زیر را طوری تغییر دهید که خروجی 200شود:

```
x = 100
def change():
    x = 200
change()
print(x)
```

تمرین ۵Function Inside Function

خروجی را مشخص کنید:

```
def outer():
   x = 50
   def inner():
      print(x)
   inner()
outer()
```

سؤال :چرا inner()می‌تواند xرا ببیند؟

تمرین ۶Nonlocal

کد زیر را طوری تغییر دهید که خروجی 200شود:

```
def outer():
   x = 100
   def inner():
      x = 200
   inner()
   print(x)
outer()
```

تمرین ۷LEGB

خروجی چیست؟

```
x = "Global"
def outer():
   x = "Enclosing"
```

```
    def inner():
        x = "Local"
        print(x)
    inner()
outer()
```

تمرین ۸ LEGB — بدونLocal

خروجی چیست؟

```
x = "Global"
def outer():
    x = "Enclosing"
    def inner():
        print(x)
    inner()
outer()
```

تمرین ۹ - تشخیصScope

مشخص کنید هر متغیر Localاست یا Global:

```
name = "Ali"
def student():
    age = 20
    print(name)
    print(age)
student()
```

تمرین ۱۰ پروژه

برنامه‌ای بنویسید که تعداد دانش‌آموزان را نگهداری کند:

```
students = 0
```

تابعی به نام:

```
add_student()
```

ایجاد کنید که هر بار اجرا شد، تعداد دانش‌آموزان را یکی افزایش دهد

مثلاً:

```
add_student()
add_student()
add_student()
print(students)
```

خروجی:

```
3
```

نکته :برای این تمرین باید مفهوم globalرا استفاده کنید

Python Decorators

Decoratorچیست؟

Decoratorابزاری است که به ما اجازه می‌دهد بدون تغییر دادن کد اصلی یک تابع، رفتار جدیدی به آن اضافه کنیم

به زبان خیلی ساده:

یک تابع که یک تابع دیگر را می‌گیرد و نسخه‌ی تغییریافته‌ای از آن را برمی‌گرداند = Decorator

مثلاً فرض کنید این تابع را داریم:

```
def hello():
   print("Hello")
```

می‌خواهیم قبل از اجرای آن یک پیام هم نمایش داده شود، بدون اینکه کد ()helloرا تغییر دهیم

Decoratorبرای همین کار مناسب است

ساده‌ترینDecorator

ابتدا یک Decorator می‌سازیم:

```
def decorator(func):
```

```
    def wrapper():
        print("Before function")
        func()
        print("After function")
    return wrapper
```

حالا آن را روی تابع hello قرار می‌دهیم:

```
@decorator
def hello():
    print("Hello")
hello()
```

خروجی:

```
Before function
Hello
After function
```

@decorator یعنی چه؟

این قسمت:

```
@decorator
def hello():
    print("Hello")
```

تقریباً معادل این است:

```
def hello():
    print("Hello")
hello = decorator(hello)
```

یعنی Python تابع hello را به Decorator می‌دهد

مثلاً:

```
def decorator(func):
    def wrapper():
        print("Start")
        func()
        print("End")
    return wrapper
```

در اینجا:

decorator → Decorator

func → تابع اصلی

wrapper → تابع جدید

مثال ساده‌تر با تغییر خروجی

Decoratorمی‌تواند خروجی تابع را تغییر دهد

```
def uppercase(func):
    def wrapper():
        return func().upper()
    return wrapper
```

حالا:

```
@uppercase
def greeting():
    return "hello ali"
print(greeting())
```

خروجی:

HELLO ALI

تابع اصلی:

```
return "hello ali"
```

است، اما Decorator خروجی را به حروف بزرگ تبدیل می‌کند

Decoratorبرای چند تابع

یک Decorator می‌تواند روی چند تابع استفاده شود:

```
def uppercase(func):
    def wrapper():
        return func().upper()
    return wrapper
```

حالا:

```
@uppercase
def hello():
    return "hello"
@uppercase
def welcome():
    return "welcome to python"
print(hello())
print(welcome())
```

خروجی:

```
HELLO
WELCOME TO PYTHON
```

بنابراین یک Decorator را می‌توان Reusableکرد

Decoratorبرای تابع دارایArgument

فرض کنید تابع ما argument دارد:

```
def hello(name):
    return "Hello " + name
```

Decoratorقبلی اینجا مشکل دارد، چون wrapperهیچ پارامتری ندارد

باید بنویسیم:

```
def uppercase(func):
    def wrapper(name):
        return func(name).upper()
    return wrapper
```

حالا:

```
@uppercase
def hello(name):
    return "Hello " + name
print(hello("Ali"))
```

خروجی:

```
HELLO ALI
```

مشکل روش بالا

فرض کنید یک تابع داریم:

```
def student(name, age):
    return f"{name} is {age} years old"
```

اگر بخواهیم Decorator بسازیم، باید دقیقاً بدانیم تابع چند argument دارد

اما این کار مناسب نیست:

```
def decorator(func):
    def wrapper(name, age):
        return func(name, age)
    return wrapper
```

چون ممکن است تابع دیگری داشته باشیم:

```
def product(name, price, quantity):
    ...
```

یا:

```
def calculate(a, b, c, d):
    ...
```

اینجاست که *args و **kwargs بسیار مهم می‌شوند

Decoratorبا *argsو **kwargs

روش استاندارد:

```
def decorator(func):
    def wrapper(*args, **kwargs):
        print("Function is running")
        result = func(*args, **kwargs)
        return result
    return wrapper
```

حالا تقریباً هر نوع تابعی را می‌توانیم Decorate کنیم

مثلاً:

```
@decorator
def hello(name):
    return "Hello " + name
print(hello("Ali"))
```

یا:

```
@decorator
def add(a, b):
    return a + b
```

```
print(add(10, 20))
```

یا:

```
@decorator
def student(name, age, city):
    return f"{name}, {age}, {city}"
print(student("Ali", 25, "Baku"))
```

args* و **kwargs در Decorator

***args**

تمام positional arguments را جمع می‌کند:

```
def wrapper(*args):
    print(args)
```

مثلاً:

```
hello("Ali", 25, "Baku")
```

می‌تواند تبدیل شود به:

```
args = ("Ali", 25, "Baku")
```

****kwargs**

تمام keyword arguments را جمع می‌کند:

```
def wrapper(**kwargs):
    print(kwargs)
```

مثلاً:

```
student(name="Ali", age=25)
```

تبدیل می‌شود به:

```
kwargs = {
    "name": "Ali",
```

```
    "age": 25
}
```

یک مثال کاربردی: زمان اجرای تابع

Decoratorها در برنامه‌های واقعی بسیار کاربردی هستند مثلاً می‌توانیم قبل و بعد از اجرای تابع پیام نمایش دهیم:

```
def log_function(func):
    def wrapper(*args, **kwargs):
        print("Function started")
        result = func(*args, **kwargs)
        print("Function finished")
        return result
    return wrapper
```

استفاده:

```
@log_function
def add(a, b):
    return a + b
result = add(10, 20)
print("Result:", result)
```

خروجی:

```
Function started
Function finished
Result: 30
```

مثال: بررسی دسترسی

Decoratorها در سیستم‌های واقعی برای Authentication / Authorizationنیز استفاده می‌شوند

مثلاً:

```
def admin_only(func):
```

```
    def wrapper(username):
        if username == "admin":
            return func(username)
        else:
            return "Access Denied"
    return wrapper
```

حالا:

```
@admin_only
def dashboard(username):
    return "Welcome to dashboard"
```

استفاده:

```
print(dashboard("admin"))
print(dashboard("Ali"))
```

خروجی:

```
Welcome to dashboard
Access Denied
```

Decoratorبا Argument

گاهی خود Decoratorهم باید argument دریافت کند مثلاً می‌خواهیم مشخص کنیم چند بار یک تابع اجرا شود

اینجا ساختار کمی پیچیده‌تر می‌شود:

```
def repeat(n):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for i in range(n):
                func(*args, **kwargs)
```

```
        return wrapper
    return decorator
```

استفاده:

```
@repeat(3)
def hello():
    print("Hello")
```

حالا:

```
hello()
```

خروجی:

```
Hello
Hello
Hello
```

چند Decorator برای یک تابع

می‌توانیم چند Decorator داشته باشیم:

```
def uppercase(func):
    def wrapper():
        return func().upper()
    return wrapper
def greeting(func):
    def wrapper():
        return "Hello " + func()
    return wrapper
```

حالا:

```
@uppercase
```

```
@greeting
def name():
   return "Ali"
```

و:

```
print(name())
```

خروجی:

```
HELLO ALI
```

functools.wraps

یک نکته مهم وجود دارد وقتی تابع را Decorate می‌کنیم، ممکن است اطلاعات اصلی تابع مثل __name__از بین برود.

مثلاً:

```
def decorator(func):
   def wrapper():
      return func()
   return wrapper
```

حالا:

```
@decorator
def hello():
   return "Hello"
```

اگر بنویسیم:

```
print(hello.__name__)
```

ممکن است ببینیم:

```
wrapper
```

در حالی که اسم تابع اصلی hello بوده است

استفاده از functools.wraps

راه استاندارد:

```
from functools import wraps
def decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
```

حالا:

```
@decorator
def hello():
    return "Hello"
```

و:

```
print(hello.__name__)
```

خروجی:

```
hello
```

@wraps(func)باعث می‌شود metadata تابع اصلی حفظ شود

چند کاربرد مهمDecorator

Decoratorها در پروژه‌های واقعی بسیار مهم هستند

برخی کاربردها:

کاربرد	مثال
Logging	ثبت اجرای توابع
Authentication	بررسی ورود کاربر
Authorization	بررسی سطح دسترسی
Timing	اندازه‌گیری زمان اجرا
Validation	بررسی ورودی‌ها
Caching	ذخیره نتایج
Debugging	پیدا کردن خطا
Web Frameworks	استفاده گسترده در Django وFlask

مثلاً در Django باDecorator ها زیاد مواجه خواهید شد

تمرین ۱

Decoratorای بنویسید که قبل از اجرای تابع بنویسد:

Starting...

و بعد از اجرای تابع بنویسد:

Finished!

مثلاً:

```
@my_decorator
def hello():
    print("Hello")
```

خروجی:

```
Starting...
Hello
Finished!
```

تمرین ۲Uppercase

Decoratorای بنویسید که خروجی هر تابع را به حروف بزرگ تبدیل کند

```
@uppercase
def message():
   return "python is easy"
```

خروجی مورد انتظار:

```
PYTHON IS EASY
```

تمرین ۳*args*

Decoratorای بنویسید که بتواند روی تابع زیر کار کند:

```
@decorator
def add(a, b):
   return a + b
```

و خروجی را قبل از return نمایش دهد:

```
Result: 30
```

تمرین ۴Logging

Decoratorای بنویسید که نام تابع را چاپ کند:

```
@log
def hello():
   print("Hello")
```

خروجی:

```
Calling function: hello
Hello
```

راهنمایی:

از این استفاده کنید:

```
func.__name__
```

تمرین ۵Access Control

Decoratorای بنویسید که فقط اگر username برابر "admin"بود اجازه اجرای تابع را بدهد

```
@admin_only
def dashboard(username):
    return "Welcome to dashboard"
```

مثلاً:

```
print(dashboard("admin"))
```

باید بنویسد:

```
Welcome to dashboard
```

ولی:

```
print(dashboard("Ali"))
```

باید بنویسد:

```
Access Denied
```

تمرین ۶Repeat

Decoratorای بنویسید که تعداد دفعات اجرای تابع را مشخص کند:

```
@repeat(5)
def hello():
    print("Hello")
```

خروجی:

```
Hello
Hello
Hello
Hello
Hello
```

تمرین ۷ - محاسبه زمان اجرا

Decoratorای بنویسید که زمان شروع و پایان اجرای یک تابع را اندازه‌گیری کند

راهنمایی:

import time

و از:

time.time()

استفاده کنید

Python Lambda Functions

Lambda Functionچیست؟

lambdaیک تابع کوچک و بدون نام (anonymous) است که معمولاً برای انجام یک کار ساده استفاده می‌شود

تابع معمولی:

```
def square(x):
    return x * x
print(square(5))
```

همین تابع با lambda:

```
square = lambda x: x * x
print(square(5))
```

خروجی:

25

پس می‌توانیم بگوییم:

def function → تابع معمولی و چندخطی

lambda → تابع کوتاه و معمولاً یک‌خطی

Syntax

ساختار Lambda به شکل زیر است:

```
lambda arguments: expression
```

مثلاً:

```
lambda x: x + 10
```

یعنی یک مقدار xدریافت کن و 10به آن اضافه کن

مثال:

```
add_ten = lambda x: x + 10
print(add_ten(5))
print(add_ten(20))
```

خروجی:

```
15
30
```

Lambdaبا یکArgument

```
square = lambda x: x * x
print(square(4))
```

خروجی:

```
16
```

مثال دیگر:

```
double = lambda x: x * 2
print(double(7))
```

خروجی:

```
14
```

Lambdaبا دوArgument

Lambdaمی‌تواند چندین ورودی داشته باشد

```
multiply = lambda x, y: x * y
print(multiply(5, 4))
```

خروجی:

20

مثال:

```
add = lambda a, b: a + b
print(add(10, 20))
```

خروجی:

30

Lambdaبا سهArgument

```
sum_numbers = lambda a, b, c: a + b + c
print(sum_numbers(10, 20, 30))
```

خروجی:

60

مقایسه def وlambda

با def

```
def square(x):
   return x * x
print(square(5))
```

با lambda

```
square = lambda x: x * x
```

```
print(square(5))
```

هر دو خروجی زیر را تولید می‌کنند:

25

بنابراین Lambda جایگزین تمام توابع نیست برای توابع پیچیده بهتر است از defاستفاده کنیم

Lambdaو شرطif

در Lambda نمی‌توانیم به شکل معمول چند خط کد بنویسیم، اما می‌توانیم از شرط کوتاه استفاده کنیم

مثلاً:

```
check = lambda x: "Even" if x % 2 == 0 else "Odd"
print(check(10))
print(check(7))
```

خروجی:

Even

Odd

یعنی:

اگر x زوج بود → Even

در غیر این صورت → Odd

Lambdaداخل یکFunction

یکی از کاربردهای مهم Lambda زمانی است که یک تابع، تابع دیگری را تولید کند

مثال:

```
def multiplier(n):
    return lambda x: x * n
```

حالا:

```
double = multiplier(2)
```

```
triple = multiplier(3)
print(double(10))
print(triple(10))
```

خروجی:

```
20
30
```

اینجا:

```
double = multiplier(2)
```

یعنی تابعی بساز که:

$x \times 2$

و:

```
triple = multiplier(3)
```

یعنی تابعی بساز که:

$x \times 3$

Lambdaو ()map

یکی از مهم‌ترین کاربردهای Lambda استفاده همراه با ()mapاست فرض کنید داریم:

```
numbers = [1, 2, 3, 4, 5]
```

می‌خواهیم همه اعداد را دو برابر کنیم

بدون:Lambda

```
def double(x):
    return x * 2
result = list(map(double, numbers))
print(result)
```

خروجی:

[2, 4, 6, 8, 10]

با:Lambda

```
numbers = [1, 2, 3, 4, 5]
result = list(map(lambda x: x * 2, numbers))
print(result)
```

خروجی:

[2, 4, 6, 8, 10]

مفهوم ()map

map(function, iterable) یعنی تابع را روی تک‌تک عناصر یک مجموعه اجرا کن

Lambdaو ()filter

()filterبرای فیلتر کردن داده‌ها استفاده می‌شود مثلاً فقط اعداد زوج را می‌خواهیم:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]
result = list(filter(lambda x: x % 2 == 0, numbers))
print(result)
```

خروجی:

[2, 4, 6, 8]

اینجا:

```
lambda x: x % 2 == 0
```

برای هر عدد بررسی می‌کند آیا عدد زوج است؟ اگر Trueباشد، عدد در نتیجه قرار می‌گیرد

مثال filter برای اعداد بزرگ‌تر از ۵

```
numbers = [2, 7, 3, 9, 1, 10, 4]
result = list(filter(lambda x: x > 5, numbers))
print(result)
```

خروجی:

[7, 9, 10]

Lambdaو sorted()

یکی دیگر از کاربردهای مهمLambda ، مرتب‌سازی است.

فرض کنید:

```
students = [
   ("Ali", 20),
   ("Sara", 18),
   ("Reza", 15)
]
```

می‌خواهیم دانش‌آموزان را بر اساس نمره مرتب کنیم.

```
students = [
   ("Ali", 20),
   ("Sara", 18),
   ("Reza", 15)
]
result = sorted(students, key=lambda x: x[1])
print(result)
```

خروجی:

```
[('Reza', 15), ('Sara', 18), ('Ali', 20)]
```

چرا x[1]؟ چون هر عضو یک Tuple است:

("Ali", 20)

↑ ↑

x[0] x[1]

پس key=lambda x: x[1] یعنی برای مرتب‌سازی، مقدار دوم هر Tuple را در نظر بگیر

مرتب‌سازی بر اساس نام

```
students = [
   ("Ali", 20),
   ("Sara", 18),
   ("Reza", 15)
]
result = sorted(students, key=lambda x: x[0])
print(result)
```

مرتب‌سازی رشته‌ها بر اساس طول

```
words = ["apple", "cat", "banana", "dog"]
result = sorted(words, key=lambda x: len(x))
print(result)
```

خروجی:

```
['cat', 'dog', 'apple', 'banana']
```

زیرا:

cat → 3

dog → 3

apple → 5

banana → 6

Lambda با Dictionary

فرض کنید:

```
students = [
   {"name": "Ali", "age": 20},
   {"name": "Sara", "age": 18},
   {"name": "Reza", "age": 22}
```

```
]
```

می‌خواهیم بر اساس سن مرتب کنیم:

```
result = sorted(students, key=lambda x: x["age"])
print(result)
```

یک مثال واقعی‌تر

فرض کنید قیمت محصولات را داریم:

```
products = [
   ("Laptop", 1200),
   ("Mouse", 25),
   ("Keyboard", 50),
   ("Monitor", 300)
]
```

مرتب‌سازی بر اساس قیمت:

```
products = [
   ("Laptop", 1200),
   ("Mouse", 25),
   ("Keyboard", 50),
   ("Monitor", 300)
]
result = sorted(products, key=lambda x: x[1])
print(result)
```

سه تابع مهم باLambda

برای دانش‌آموزان می‌توانید این سه مورد را به‌عنوان یک جدول بسیار مهم ارائه کنید:

Function	کاربرد	مثال
map()	تغییر عناصر	دو برابر کردن اعداد
filter()	انتخاب عناصر	انتخاب اعداد زوج
sorted()	مرتب‌سازی	مرتب کردن بر اساس سن

مثلاً:

```
numbers = [1, 2, 3, 4, 5]

# map

a = list(map(lambda x: x * 2, numbers))

# filter

b = list(filter(lambda x: x % 2 == 0, numbers))

# sorted

c = sorted(numbers, key=lambda x: -x)

print(a)

print(b)

print(c)
```

تمرین ۱

یک Lambda بنویسید که عدد را دریافت کرده و مربع آن را برگرداند

مثلاً:

```
Input: 5

Output: 25
```

تمرین ۲

یک Lambda بنویسید که دو عدد را دریافت کند و عدد بزرگ‌تر را برگرداند

راهنمایی:

lambda a, b: ...

تمرین ۳

با استفاده از lambdaو map()تمام اعداد لیست زیر را سه برابر کنید:

numbers = [1, 2, 3, 4, 5]

نتیجه:

[3, 6, 9, 12, 15]

تمرین ۴

با استفاده از lambdaو filter()فقط اعداد زوج را نمایش دهید:

numbers = [12, 7, 9, 20, 15, 8, 30]

تمرین ۵

با استفاده از lambdaو filter()فقط اعداد بزرگ‌تر از 10را پیدا کنید:

numbers = [5, 12, 3, 20, 8, 15, 2]

تمرین ۶

لیست زیر را بر اساس طول کلمات مرتب کنید:

words = ["Python", "AI", "Programming", "Code", "Data"]

از sorted()و lambdaاستفاده کنید

تمرین ۷

دانش‌آموزان را بر اساس نمره مرتب کنید:

```
students = [
   ("Ali", 18),
   ("Sara", 20),
   ("Reza", 15),
   ("Maryam", 19)
]
```

تمرین ۸

با استفاده از lambdaو filter()اعداد زوج و بزرگ‌تر از 10را پیدا کنید:

```
numbers = [5, 12, 8, 20, 15, 24, 7, 30]
```

خروجی مورد انتظار:

```
[12, 20, 24, 30]
```

تمرین ۹

لیست زیر را بر اساس قیمت از کم به زیاد مرتب کنید:

```
products = [
   ("Laptop", 1200),
   ("Mouse", 25),
   ("Keyboard", 50),
   ("Monitor", 300)
]
```

تمرین ۱۰

برنامه‌ای بنویسید که با استفاده از map()و lambdaقیمت محصولات زیر را ۱۰ درصد افزایش دهد:

```
prices = [100, 250, 500, 1000]
```

خروجی:

```
[110, 275, 550, 1100]
```

Python Recursion

Recursionچیست؟

Recursionبازگشت یعنی یک تابع، خودش را دوباره صدا بزند.

مثلاً:

```
def hello():
   print("Hello")
```

```
hello()
```

این برنامه مشکل دارد، چون تابع هیچ‌وقت متوقف نمی‌شود و دائماً خودش را صدا می‌زند بنابراین در Recursion یک موضوع بسیار مهم داریم: **تابع باید یک نقطه توقف داشته باشد**

مثال

فرض کنید می‌خواهیم از عدد ۵ تا ۱ شمارش معکوس کنیم

بدون:Recursion

```
for i in range(5, 0, -1):
    print(i)
```

با Recursion:

```
def countdown(n):
    if n == 0:
        print("Done!")
    else:
        print(n)
        countdown(n - 1)
countdown(5)
```

خروجی:

```
5
4
3
2
1
Done!
```

این تابع چگونه کار می‌کند؟

وقتی می‌نویسیم:

countdown(5)

تابع اجرا می‌شود:

countdown(5)

↓

print(5)

↓

countdown(4)

↓

print(4)

↓

countdown(3)

↓

print(3)

↓

countdown(2)

↓

print(2)

↓

countdown(1)

↓

print(1)

↓

countdown(0)

↓

Done!

بنابراین:

```
countdown(n - 1)
```

باعث می‌شود تابع دوباره اجرا شود، ولی این بار با عدد کوچک‌تر

Base Caseچیست؟

در هر تابع Recursive معمولاً دو قسمت مهم داریم:

Base Case

شرطی که باعث می‌شود Recursion متوقف شود

مثلاً:

```
if n == 0:
    print("Done!")
```

Recursive Case

قسمتی که تابع خودش را دوباره صدا می‌زند:

```
countdown(n - 1)
```

مثالFactorial

یکی از معروف‌ترین مثال‌هایRecursion ، محاسبه Factorial است

می‌دانیم: $5! = 5 \times 4 \times 3 \times 2 \times 1$ یعنی $5! = 120$ ، در ریاضی $n! = n \times (n-1)!$

پس می‌توانیم آن را با Recursion بنویسیم:

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)
print(factorial(5))
```

خروجی:

```
120
```

Factorial چگونه اجرا می‌شود؟

وقتی می‌نویسیم:

factorial(5)

در ابتدا:

factorial(5)

= 5 × factorial(4)

بعد:

= 5 × 4 × factorial(3)

بعد:

= 5 × 4 × 3 × factorial(2)

بعد:

= 5 × 4 × 3 × 2 × factorial(1)

و در نهایت:

factorial(0) = 1

بنابراین:

5 × 4 × 3 × 2 × 1

= 120

یک مثال بسیار ساده‌تر برای درک Recursion

می‌توانیم تابعی بنویسیم که اعداد ۱ تا nرا چاپ کند:

```
def count(n):
    if n == 0:
        return
    count(n - 1)
    print(n)
```

```
count(5)
```

خروجی:

```
1
2
3
4
5
```

دقت کنید اینجا print()بعد از فراخوانی Recursive قرار گرفته است این مثال برای نشان دادن تفاوت ترتیب اجرای Recursionبسیار مفید است

Fibonacci

دنباله Fibonacci به شکل زیر است:

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

هر عدد از جمع دو عدد قبلی به دست می‌آید:

0 + 1 = 1

1 + 1 = 2

1 + 2 = 3

2 + 3 = 5

...

می‌توانیم آن را با Recursion بنویسیم:

```
def fibonacci(n):
  if n <= 1:
    return n
  else:
    return fibonacci(n - 1) + fibonacci(n - 2)
```

```
print(fibonacci(7))
```

خروجی:

```
13
```

البته این پیاده‌سازی Fibonacci برای ورودی‌های بزرگ کند است و بیشتر برای آموزش مفهوم Recursion مناسب است

Recursionبا List

Recursionمی‌تواند برای پردازش عناصر یک List نیز استفاده شود

مثلاً مجموع عناصر:

```
def sum_list(numbers):
    if len(numbers) == 0:
        return 0
    else:
        return numbers[0] + sum_list(numbers[1:])
numbers = [1, 2, 3, 4, 5]
print(sum_list(numbers))
```

خروجی:

```
15
```

ایده اصلی:

[1, 2, 3, 4, 5]

↓

1 + sum([2, 3, 4, 5])

↓

1 + 2 + sum([3, 4, 5])

↓

1 + 2 + 3 + sum([4, 5])

↓

...

اشتباه رایجInfinite Recursion :

این کد را ببینید:

```
def test(n):
    print(n)
    test(n)
test(5)
```

چه اتفاقی می‌افتد؟

تابع:

test(5)

↓

test(5)

↓

test(5)

↓

test(5)

↓

...

هیچ‌وقت به پایان نمی‌رسد در نهایت Python خطایی مانند زیر می‌دهد:

```
RecursionError: maximum recursion depth exceeded
```

Recursionو Loop

بسیاری از کارهایی که با Recursion انجام می‌دهیم، با Loop هم قابل انجام هستند

مثلاً:

با while

```
i = 5
while i > 0:
    print(i)
    i -= 1
```

باRecursion

```
def countdown(i):
    if i == 0:
        return
    print(i)
    countdown(i - 1)
countdown(5)
```

بنابراین سؤال مهم این است چه زمانی Recursion استفاده کنیم؟

Recursionدر مسائلی بسیار مفید است که ساختار مسئله ذاتاً تکرارشونده یا تو در تو باشد؛ مثلاً:

- ساختارهای درختی (Tree)
- پیمایش پوشه‌ها و زیرپوشه‌ها
- بعضی الگوریتم‌های Divide and Conquer
- DFS
- بعضی مسائل ریاضی
- Backtracking

برای یک شمارش ساده معمولاً forیا whileانتخاب بهتری است

Recursion Depth

Pythonبرای جلوگیری از Recursion بیش از حد، محدودیتی برای عمق فراخوانی‌های Recursive دارد

می‌توانیم این مقدار را ببینیم:

```
import sys
print(sys.getrecursionlimit())
```

معمولاً مقداری در حدود 1000است، ولی مقدار دقیق به محیط Python بستگی دارد

مهم‌ترین الگوی ذهنی:

```
def function(n):
   if stop_condition:
      return result
   return function(smaller_problem)
```

تمرین ۱

تابعی بنویسید که از 10تا 1را با Recursion چاپ کند

تمرین ۲

تابعی بنویسید که از 1تا 10را با Recursion چاپ کند

تمرین ۳

تابعی بنویسید که Factorial یک عدد را محاسبه کند:

Input: 5

Output: 120

تمرین ۴

تابعی بنویسید که مجموع اعداد 1تا nرا با Recursion محاسبه کند:

Input: 5

Output: 15

تمرین ۵

تابعی بنویسید که مجموع عناصر List زیر را با Recursion محاسبه کند:

```
numbers = [10, 20, 30, 40]
```

تمرین ۶

تابعی بنویسید که مشخص کند یک عدد مثبت زوج است یا فرد و این کار را با Recursion انجام دهید

تمرین ۷

تابعی برای محاسبه توان بنویسید:

```
power(2, 5)
```

باید نتیجه زیر را بدهد:

```
32
```

تمرین ۸

تابعی بنویسید که یک String را با Recursion برعکس کند:

```
Input: "Python"
Output: "nohtyP"
```

Python Generators

Generatorچیست؟

Generatorنوعی تابع است که می‌تواند اجرای خودش را متوقف کند و بعداً از همان نقطه ادامه دهد در Generator به جای returnاز کلمه‌ی کلیدی yieldاستفاده می‌کنیم

```
def my_generator():
  yield 1
  yield 2
  yield 3
```

حالا می‌توانیم روی Generator با forحرکت کنیم:

```
for value in my_generator():
  print(value)
```

خروجی:

```
1
2
3
```

نکته مهم : وقتی تابع Generator را صدا می‌زنیم، کدهای داخل آن بلافاصله اجرا نمی‌شوند

```
gen = my_generator()
print(gen)
```

در اینجا gen یک generator object است

تفاوت return و yield

با return

```
def numbers():
    return 1
    return 2
    return 3
```

فقط 1 برگردانده می‌شود، چون return اجرای تابع را تمام می‌کند

با yield

```
def numbers():
    yield 1
    yield 2
    yield 3
```

در این حالت:

1

↓

pause

↓

2

↓

pause

↓

3

یعنی yieldتابع را متوقف می‌کند، ولی تابع را کاملاً تمام نمی‌کند

استفاده از for

ساده‌ترین روش استفاده ازGenerator ، حلقه forاست

```
def count_up_to(n):
    count = 1
    while count <= n:
        yield count
        count += 1
for number in count_up_to(5):
    print(number)
```

خروجی:

```
1
2
3
4
5
```

next()چیست؟

با next()می‌توانیم Generator را مرحله به مرحله اجرا کنیم

```
def numbers():
    yield 10
    yield 20
    yield 30
gen = numbers()
print(next(gen))
print(next(gen))
```

```
print(next(gen))
```

خروجی:

```
10
20
30
```

هر بار که next()را اجرا می‌کنیم، Generatorتا yieldبعدی جلو می‌رود

Generatorو Memory

یکی از مهم‌ترین مزایای Generator این است که همه‌ی داده‌ها را یکجا در حافظه نگه نمی‌دارد فرض کنید می‌خواهیم اعداد 0 تا 1,000,000را تولید کنیم

روش معمول

```
numbers = [x for x in range(1000000)]
```

در این حالت یک listبزرگ ایجاد می‌شود

باGenerator

```
numbers = (x for x in range(1000000))
```

اعداد به صورت تدریجی تولید می‌شوند مثلاً:

```
numbers = (x for x in range(1000000))
print(next(numbers))
print(next(numbers))
print(next(numbers))
```

خروجی:

```
0
1
2
```

همه‌ی یک میلیون مقدار از ابتدا ساخته نشده‌اند

نتیجه: List همه داده‌ها ⟵ حافظه

Generator داده فعلی ⟵ حافظه داده بعدی ⟵ هنگام نیاز تولید می‌شود

Generator Expression

مشابه List Comprehensionمی‌توانیم Generator Expression داشته باشیم

List Comprehension

```
numbers = [x * 2 for x in range(5)]
print(numbers)
```

خروجی:

```
[0, 2, 4, 6, 8]
```

Generator Expression

```
numbers = (x * 2 for x in range(5))
print(numbers)
```

اینجا به جای []از ()استفاده شده است

برای دیدن مقادیر:

```
print(list(numbers))
```

خروجی:

```
[0, 2, 4, 6, 8]
```

یک مثال کاربردی

فرض کنید می‌خواهیم قیمت محصولات را یکی‌یکی تولید کنیم:

```
def prices():
    yield 100
    yield 250
    yield 500
    yield 750
```

```
for price in prices():
    print("Price:", price)
```

خروجی:

```
Price: 100
Price: 250
Price: 500
Price: 750
```

Generatorبرای اعداد زوج

```
def even_numbers(n):
    for i in range(1, n + 1):
        if i % 2 == 0:
            yield i
for number in even_numbers(10):
    print(number)
```

خروجی:

```
2
4
6
8
10
```

Generatorبرای اعداد فرد

```
def odd_numbers(n):
    for i in range(1, n + 1):
```

```
        if i % 2 != 0:
            yield i
for number in odd_numbers(10):
    print(number)
```

خروجی:

```
1
3
5
7
9
```

Generatorبرای مربع اعداد

```
def squares(n):
    for i in range(1, n + 1):
        yield i ** 2
for number in squares(5):
    print(number)
```

خروجی:

```
1
4
9
16
25
```

Generatorبا while

```
def countdown(n):
    while n > 0:
```

```
        yield n
        n -= 1
for number in countdown(5):
    print(number)
```

خروجی:

```
5
4
3
2
1
```

Generatorبرای Fibonacci

یک مثال جالب‌تر:

```
def fibonacci(n):
    a = 0
    b = 1
    for i in range(n):
        yield a
        a, b = b, a + b
for number in fibonacci(10):
    print(number)
```

خروجی:

```
0
1
1
2
```

```
3
5
8
13
21
34
```

مزیت این روش این است که لازم نیست تمام دنباله را از قبل در یک List ذخیره کنیم

Generatorبا ()send

این قسمت کمی پیشرفته‌تر است و برای دانشجوی مبتدی می‌توانید فعلاً فقط معرفی کنید

```
def receiver():
   while True:
      value = yield
      print("Received:", value)
gen = receiver()
next(gen)
gen.send("Hello")
gen.send("Python")
gen.send("Generator")
```

خروجی:

```
Received: Hello
Received: Python
Received: Generator
```

close()

می‌توانیم Generator را متوقف کنیم:

```
def numbers():
```

```
    yield 1
    yield 2
    yield 3
gen = numbers()
print(next(gen))
gen.close()
```

بعد از close()، Generatorمتوقف می‌شود

تفاوت مهم List وGenerator

Generator	List	ویژگی
یکی‌یکی	همه از ابتدا	ساخت داده‌ها
کمتر	بیشتر	حافظه
×	✓	[]
✓Generator Expression	×	()
✓	×	yield
✓	معمولاً×	next()
بسیار مناسب	کمتر	مناسب داده‌های بسیار بزرگ
معمولاً یک‌بارمصرف	✓	امکان پیمایش مجدد

تمرین ۱ - اعداد ۱ تا ۱۰

یک Generator به نام numbers()بنویسید که اعداد 1تا 10را تولید کند

خروجی:

```
1
2
3
...
10
```

تمرین ۲ - اعداد زوج

یک Generator بنویسید که اعداد زوج بین 1تا 20را تولید کند.

خروجی:

2

4

6

8

10

12

14

16

18

20

تمرین ۳ - مربع اعداد

یک Generator بنویسید که مربع اعداد 1تا 10را تولید کند

1

4

9

16

...

100

تمرین ۴Countdown

یک Generator بنویسید که از عددی که کاربر وارد می‌کند تا 1شمارش معکوس انجام دهد

مثلاً:

Enter number: 5

5

4

3

2

1

تمرین ۵Fibonacci

یک Generator بنویسید که 10عدد اول Fibonacci را تولید کند

0

1

1

2

3

5

8

13

21

34

تمرین ۶()next

Generatorزیر را بررسی کنید:

```
def test():
   yield "A"
   yield "B"
   yield "C"
gen = test()
```

```
print(next(gen))
print(next(gen))
```

سؤال :خروجی چیست؟

سپس یک next()دیگر اضافه کنید و نتیجه را مشاهده کنید

تمرین ۷Generator Expression

کد زیر را بنویسید: numbers = (x * 3 for x in range(1, 6)) سپس با next()مقادیر را یکی‌یکی چاپ کنید

تمرین ۸ - تشخیص عدد اول

یک Generator بنویسید که اعداد اول بین 1تا 50را یکی‌یکی تولید کند

خروجی باید شامل اعداد زیر باشد:

2

3

5

7

11

...

47

تمرین ۹ - فیلتر نمرات

لیست زیر را داریم:

```
grades = [12, 8, 19, 15, 7, 20, 10]
```

یک Generator بنویسید که فقط نمرات بزرگ‌تر یا مساوی ۱۰ را تولید کند

تمرین ۱۰ - پروژه کوچک

یک Generator برای سیستم فروشگاه بنویسید که قیمت محصولات را یکی‌یکی تولید کند:

```
prices = [120, 250, 80, 450, 300]
```

سپس با استفاده از:Generator

۱. قیمت‌ها را یکی‌یکی چاپ کنید

۲. قیمت‌های بیشتر از 200را نمایش دهید

۳. مجموع قیمت‌ها را محاسبه کنید

Function

↓

return

↓

yield

↓

Generator

↓

next()

و مهم‌ترین تفاوت را این‌طور خلاصه کنید:

return → تابع را تمام می‌کند

yield → تابع را موقتاً متوقف می‌کند

این جمله معمولاً باعث می‌شود مفهوم Generator خیلی سریع‌تر جا بیفتد